



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №11

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 WebView.....	3
1.2 Работа с мультимедиа.....	6
1.3 Анимации в Android.....	7
1.3.1 Анимация вращения элемента.....	7
1.3.2 Анимация перемещения элемента по экрану.....	9
1.3.3 Анимация изменения размера элемента.....	11
1.4 Уведомления.....	12
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	17
2.1 Просмотр веб-страницы через WebView.....	17
2.2 Реализация проигрывания музыки из интернета.....	19
2.3 Реализация различных анимаций элементов UI.....	21
2.4 Реализация отправки уведомлений: обычная и отложенная.....	23
ЗАКЛЮЧЕНИЕ.....	29

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 WebView

WebView в Android — это компонент, который позволяет разработчикам показывать веб-страницы внутри приложения. Это своего рода встроенный браузер, управляемый при помощи класса WebView. Он используется для отображения веб-контента, такого как пользовательские страницы для авторизации, справочные материалы или даже некоторые виды интерактивного контента внутри приложения.

Основные функции и использование WebView:

- отображение веб-страниц: можно загружать HTML-страницы из интернета или загружать их как локальные ресурсы;
- интерактивность: пользователи могут взаимодействовать со страницами, как в обычном веб-браузере;
- JavaScript: WebView поддерживает выполнение JavaScript, что позволяет взаимодействовать с веб-страницей и обрабатывать пользовательский ввод;
- настройка и безопасность: разработчики могут настраивать поведение WebView, включая управление кэшированием, cookies, историей просмотров и т.д. Важно правильно настроить параметры безопасности, чтобы избежать уязвимостей.

Рассмотрим пример использования WebView.

Сначала идёт добавление WebView в XML макет Activity (рисунок 1.1.1).

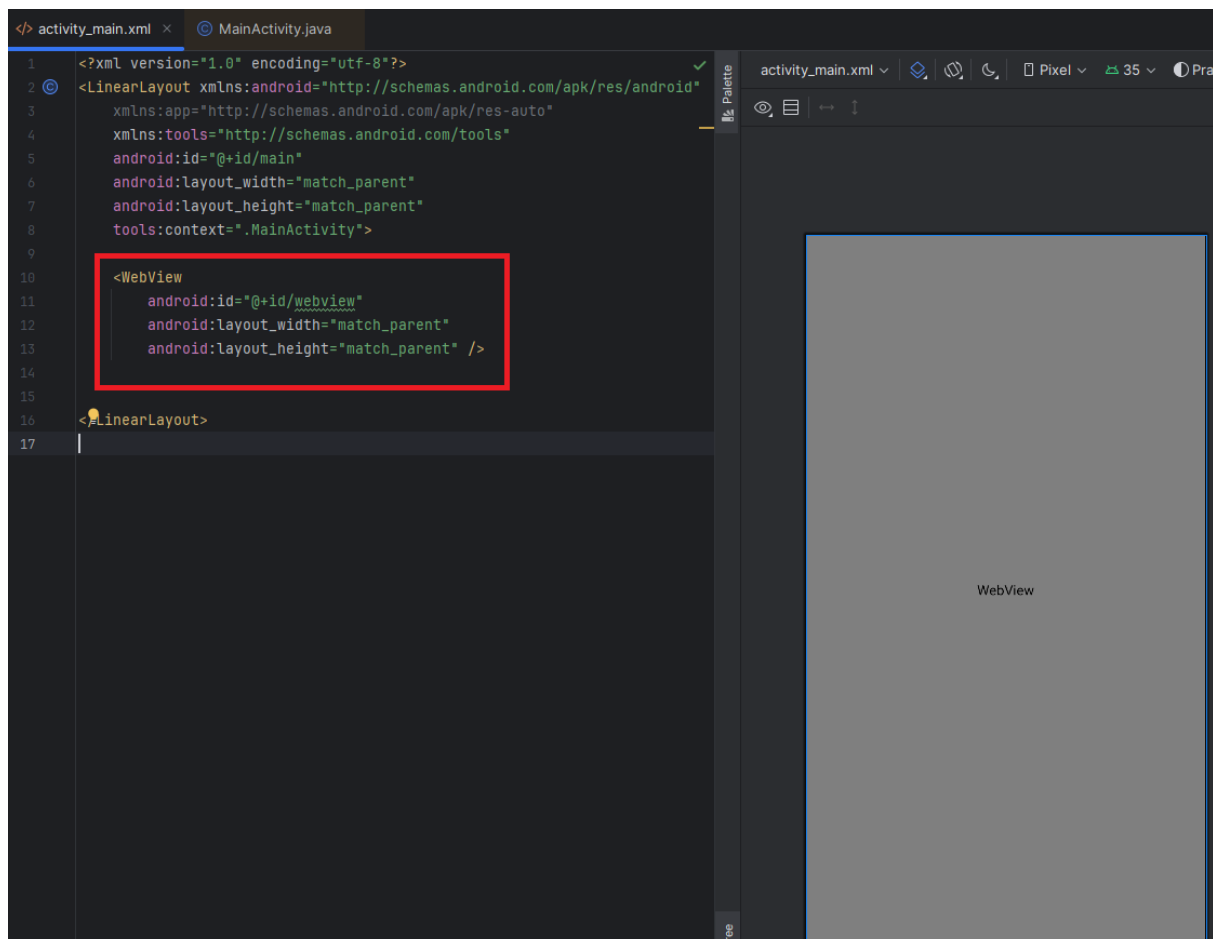


Рисунок 1.1.1 – Добавление WebView в XML макет Activity

Далее необходимо настроить WebView в коде Activity (рисунок 1.1.2).

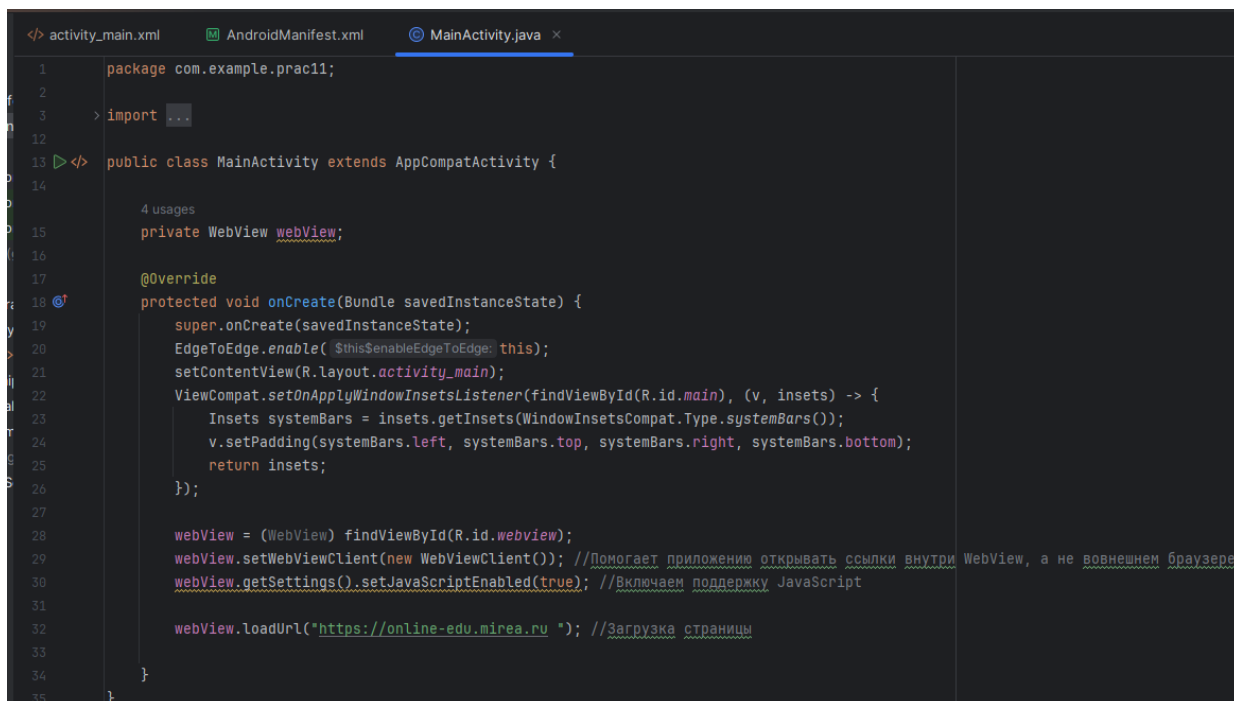
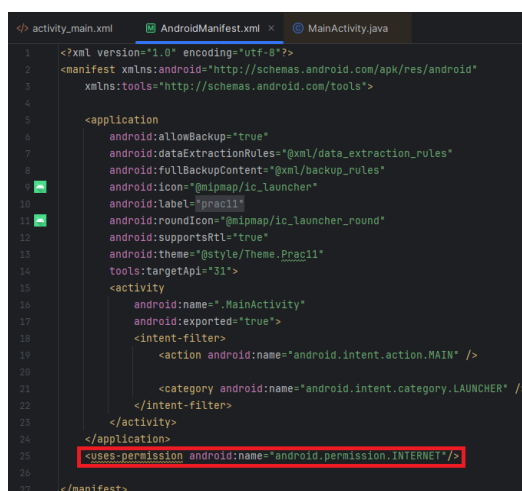


Рисунок 1.1.2 – Настройка WebView в коде Activity

Важные моменты:

- **WebViewClient**: этот класс помогает управлять загрузкой различных URL и следить за процессом загрузки веб-страницы;
- **JavaScript**: включение JavaScript необходимо для многих современных веб-страниц, которые зависят от клиентских скриптов для пользовательской интерактивности.

Для получения доступа к интернету из приложения, необходимо указать в файле манифеста **AndroidManifest.xml** соответствующее разрешение (рисунок 1.1.3).



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.Prac11"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
    <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

Рисунок 1.1.3 – Добавление разрешения на выход в интернет

При запуске приложения отобразится выбранный сайт (рисунок 1.1.4).

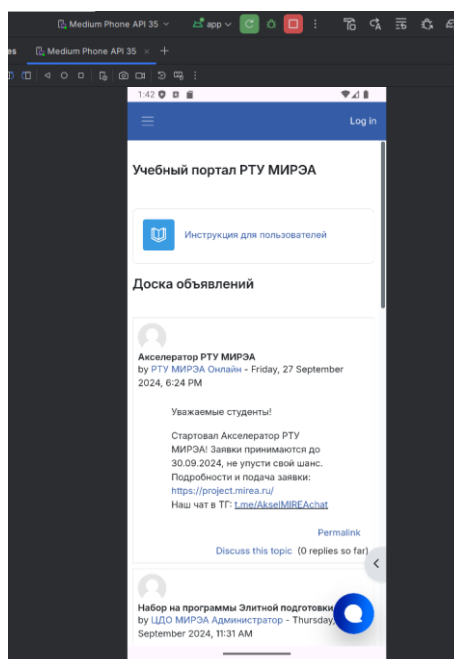
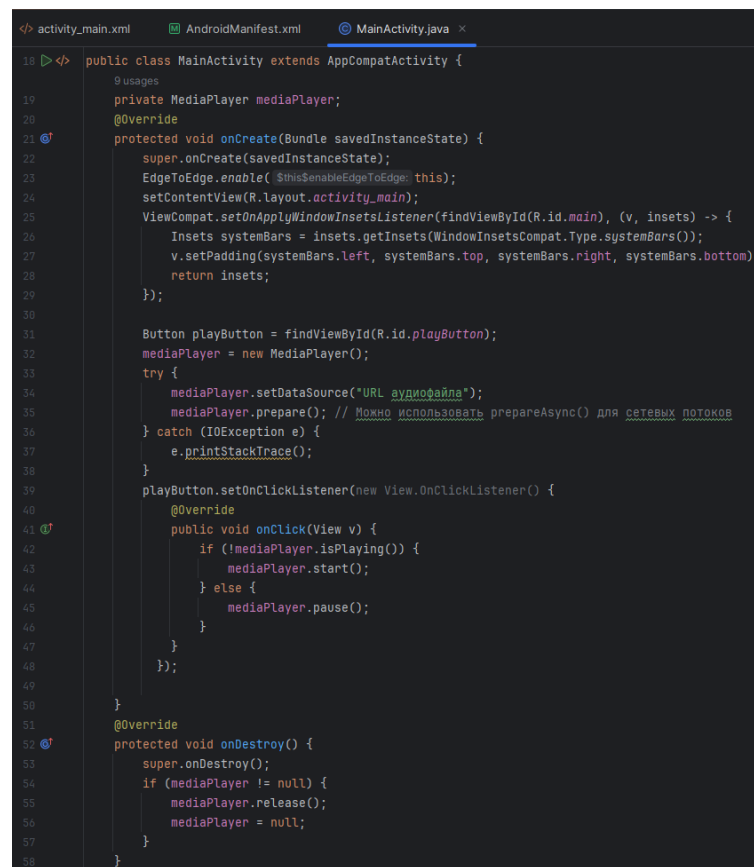


Рисунок 1.1.4 – Отображение в WebView выбранного сайта

1.2 Работа с мультимедиа

Работа с мультимедиа в Android — это важная часть разработки мобильных приложений, так как позволяет включать в приложения такие функции, как воспроизведение музыки, видео, захват изображений и видео с камеры, и обработка звука. Это улучшает интерактивность и функциональность приложения. Android предоставляет класс `MediaPlayer` для воспроизведения медиафайлов. Эти файлы могут быть локальными (хранятся на устройстве) или удаленными (доступны через интернет). Ранее был рассмотрен пример воспроизведения локального файла, теперь рассмотрим воспроизведение удаленного файла.

В код `activity` необходимо добавить логику создания и управления `MediaPlayer` (рисунок 1.2.1).



```
18 > public class MainActivity extends AppCompatActivity {
19     @SuppressWarnings("unused")
20     private MediaPlayer mediaPlayer;
21     @Override
22     protected void onCreate(Bundle savedInstanceState) {
23         super.onCreate(savedInstanceState);
24         EdgeToEdge.enable(this);
25         setContentView(R.layout.activity_main);
26         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
27             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
28             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
29             return insets;
30         });
31
32         Button playButton = findViewById(R.id.playButton);
33         mediaPlayer = new MediaPlayer();
34         try {
35             mediaPlayer.setDataSource("URL аудиофайла");
36             mediaPlayer.prepare(); // Можно использовать prepareAsync() для сетевых потоков
37         } catch (IOException e) {
38             e.printStackTrace();
39         }
40
41         playButton.setOnClickListener(new View.OnClickListener() {
42             @Override
43             public void onClick(View v) {
44                 if (!mediaPlayer.isPlaying()) {
45                     mediaPlayer.start();
46                 } else {
47                     mediaPlayer.pause();
48                 }
49             }
50         });
51
52     }
53     @Override
54     protected void onDestroy() {
55         super.onDestroy();
56         if (mediaPlayer != null) {
57             mediaPlayer.release();
58             mediaPlayer = null;
59         }
60     }
61 }
```

Рисунок 1.2.1 – Логика работы `MediaPlayer`

Этот пример демонстрирует базовый способ воспроизведения аудио с использованием `MediaPlayer`. Следует обратить внимание на обработку исключений и корректное освобождение ресурсов в методе `onDestroy()`.

Работая с мультимедиа, важно помнить о потенциальных исключениях и о необходимости управления ресурсами, чтобы избежать утечек памяти и сохранить стабильность приложения.

1.3 Анимации в Android

Анимации в Android используются для улучшения пользовательского интерфейса путём добавления визуальных эффектов при взаимодействии с элементами приложения. Они могут помочь сделать приложение более динамичным и интересным, а также интуитивно понятным, подчеркивая изменения состояний, переходы между активностями или фрагментами и реакции на действия пользователя.

Типы анимации в Android:

- View анимации: это простые анимации, которые можно применять к элементам интерфейса (views), такие как повороты, масштабирование, смещения и прозрачность. Они описываются в XML и могут быть запущены в коде;
- анимация свойств (Property Animations): эти анимации предоставляют более сложный контроль, позволяя анимировать любое свойство объекта, такое как цвет фона, позиция или размер. Наиболее популярным классом для таких анимаций является `ObjectAnimator`;
- анимации переходов: используются для анимации переходов между активностями или фрагментами, позволяя создавать сложные анимации при переключении между экранами.

Рассмотрим несколько примеров анимации элементов.

1.3.1 Анимация вращения элемента

Для анимации вращения элемента необходимо воспользоваться таким классом, как `ObjectAnimator` (рисунок 1.3.1.1).

```

0 ></> public class MainActivity extends AppCompatActivity {
1     no usages
2     private MediaPlayer mediaPlayer;
3     @Override
4     protected void onCreate(Bundle savedInstanceState) {
5         super.onCreate(savedInstanceState);
6         EdgeToEdge.enable(this);
7         setContentView(R.layout.activity_main);
8         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
9             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
10            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
11            return insets;
12        });
13
14        ImageView imageView = findViewById(R.id.rotateImageView);
15        ObjectAnimator rotateAnim = ObjectAnimator.ofFloat(imageView, "rotation", 0f, 360f);
16        rotateAnim.setDuration(2000);
17        rotateAnim.setRepeatCount(ObjectAnimator.INFINITE);
18        rotateAnim.setRepeatMode(ObjectAnimator.RESTART);
19        rotateAnim.start();
20    }
21 }

```

Рисунок 1.3.1.1 – Логика анимации вращения элемента

ObjectAnimator – это подкласс ValueAnimator, который позволяет нам устанавливать целевой объект и свойство объекта для анимации. Это простой способ изменить свойства вида с указанной продолжительностью. Мы можем указать конечную позицию и продолжительность анимации.

Методы:

- ofFloat(): создает и возвращает ObjectAnimation, который анимирует координаты;
- setDuration(): устанавливает продолжительность анимации;
- getRepeatCount() – определяет, сколько раз должна повториться анимация;
- getRepeatMode() – определяет, что должна делать эта анимация, когда она дойдет до конца.

При запуске приложения выбранный ImageView начнёт вращаться по часовой стрелке (рисунки 1.3.1.2 – 1.3.1.3).

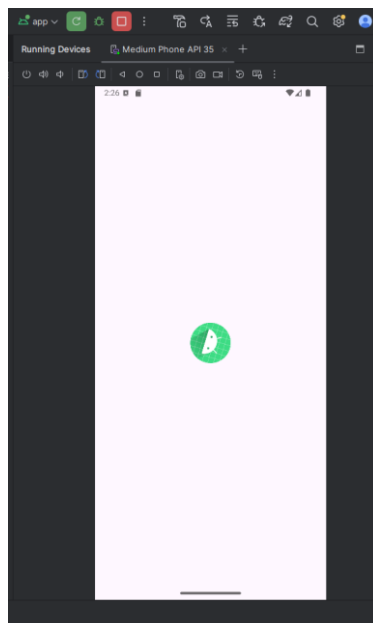


Рисунок 1.3.1.2 – Часть 1 отображения вращения ImageView

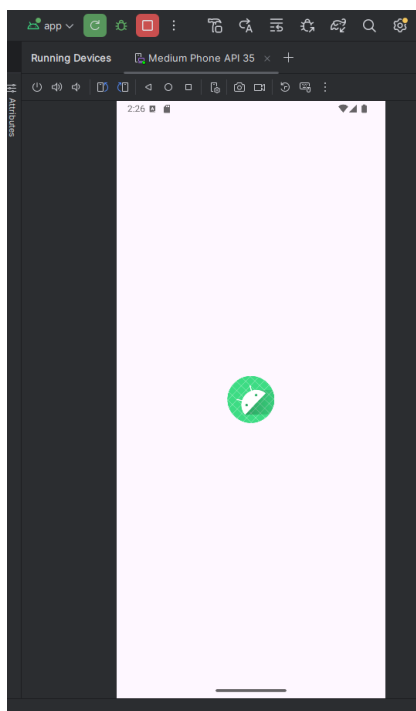


Рисунок 1.3.1.3 - Часть 2 отображения вращения ImageView

1.3.2 Анимация перемещения элемента по экрану

В данном случае также будет использоваться ObjectAnimator с методами, описанными в пункте 1.3.1 (рисунок 1.3.2.1).

```

13 import androidx.appcompat.app.AppCompatActivity;
14 import androidx.core.graphics.Insets;
15 import androidx.core.view.ViewCompat;
16 import androidx.core.view.WindowInsetsCompat;
17
18 import java.io.IOException;
19
20 public class MainActivity extends AppCompatActivity {
21     no usages
22     private MediaPlayer mediaPlayer;
23     @Override
24     protected void onCreate(Bundle savedInstanceState) {
25         super.onCreate(savedInstanceState);
26         EdgeToEdge.enable(this);
27         setContentView(R.layout.activity_main);
28         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
29             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
30             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
31             return insets;
32         });
33
34         final Button moveButton =
35             findViewById(R.id.moveButton);
36         moveButton.setOnClickListener(new View.OnClickListener() {
37             @Override
38             public void onClick(View v) {
39                 ObjectAnimator moveAnim =
40                     ObjectAnimator.ofFloat(moveButton, "translationX", 0f, 300f);
41                 moveAnim.setDuration(1000);
42                 moveAnim.start();
43             }
44         });
45     }
46 }

```

Рисунок 1.3.2.1 – Логика перемещения элемента по экрану

При нажатии на кнопку она переместится вправо на заранее прописанное расстояние (рисунки 1.3.2.2 – 1.3.2.3).

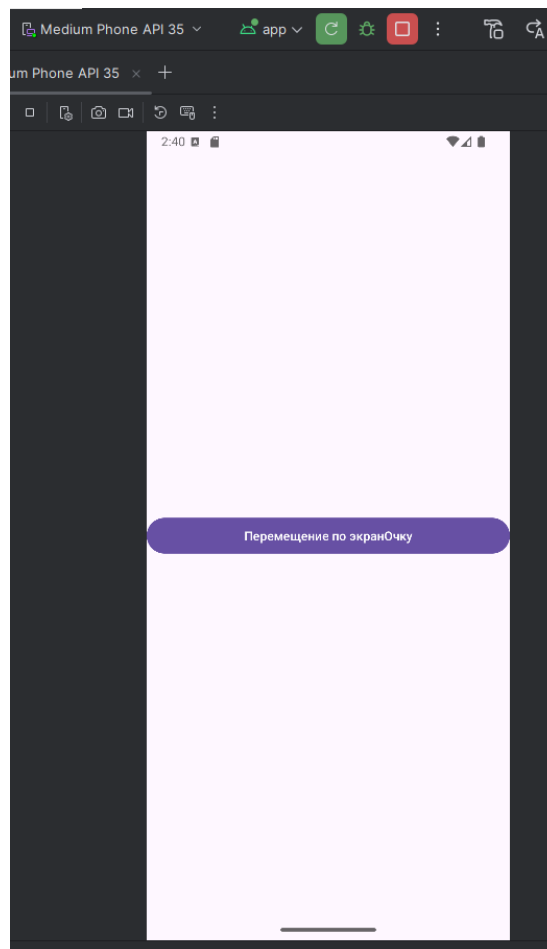


Рисунок 1.3.2.2 – Часть 1 отображения перемещения элемента

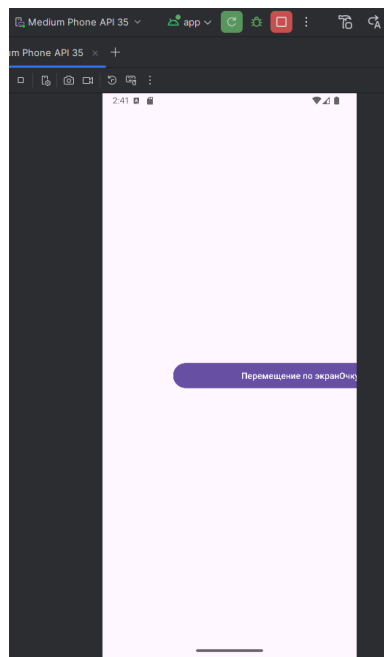


Рисунок 1.3.2.3 – Часть 2 отображения перемещения элемента

1.3.3 Анимация изменения размера элемента

И последняя на рассмотрение анимация — анимация изменения размера элемента (рисунки 1.3.3.1 – 1.3.3.3).

```

21 <> public class MainActivity extends AppCompatActivity {
22     no usages
23     private MediaPlayer mediaPlayer;
24     @Override
25     protected void onCreate(Bundle savedInstanceState) {
26         super.onCreate(savedInstanceState);
27         EdgeToEdge.enable( $this$enableEdgeToEdge: this);
28         setContentView(R.layout.activity_main);
29         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
30             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
31             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
32             return insets;
33         });
34
35         TextView scaleText = findViewById(R.id.scaleTextView);
36         scaleText.setOnClickListener(new View.OnClickListener() {
37             @Override
38             public void onClick(View v) {
39                 ObjectAnimator scaleX =
40                     ObjectAnimator.ofFloat(scaleText, propertyName: "scaleX", ...values: 1f, 2f);
41                 ObjectAnimator scaleY = ObjectAnimator.ofFloat(scaleText, propertyName: "scaleY", ...values: 1f, 2f);
42                 scaleX.setDuration(1000);
43                 scaleY.setDuration(1000);
44                 scaleX.start();
45                 scaleY.start();
46             }
47         });
48     }
49 }
50
51

```

Рисунок 1.3.3.1 – Логика анимации изменения размера элемента

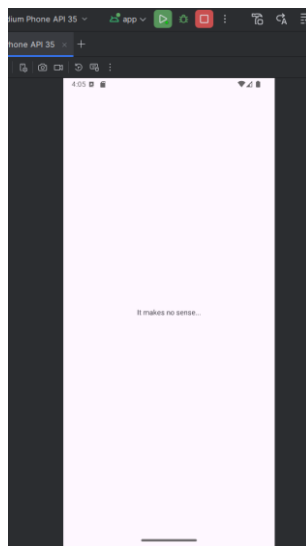


Рисунок 1.3.3.2 – Часть 1 отображения анимации изменения размера элемента

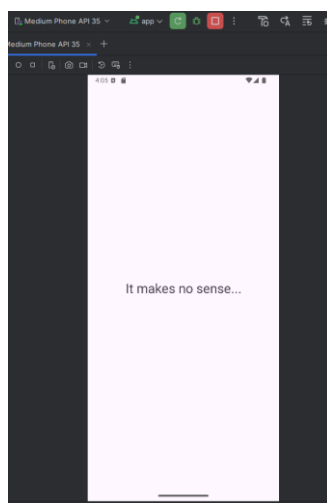


Рисунок 1.3.3.3 – Часть 2 отображения анимации изменения размера элемента

Эти примеры демонстрируют различные способы использования анимации для улучшения визуального восприятия и интерактивности Android-приложений.

Анимацию можно создавать как отдельный ресурс в папке под названием «anim» для хранения ресурсов анимации.

1.4 Уведомления

Уведомления в Android представляют собой сообщения, которые можно отображать вне интерфейса вашего приложения для информирования пользователей о различных событиях, даже когда приложение не используется активно. Это мощный инструмент для

повышения взаимодействия и поддержания актуальности приложения в глазах пользователей.

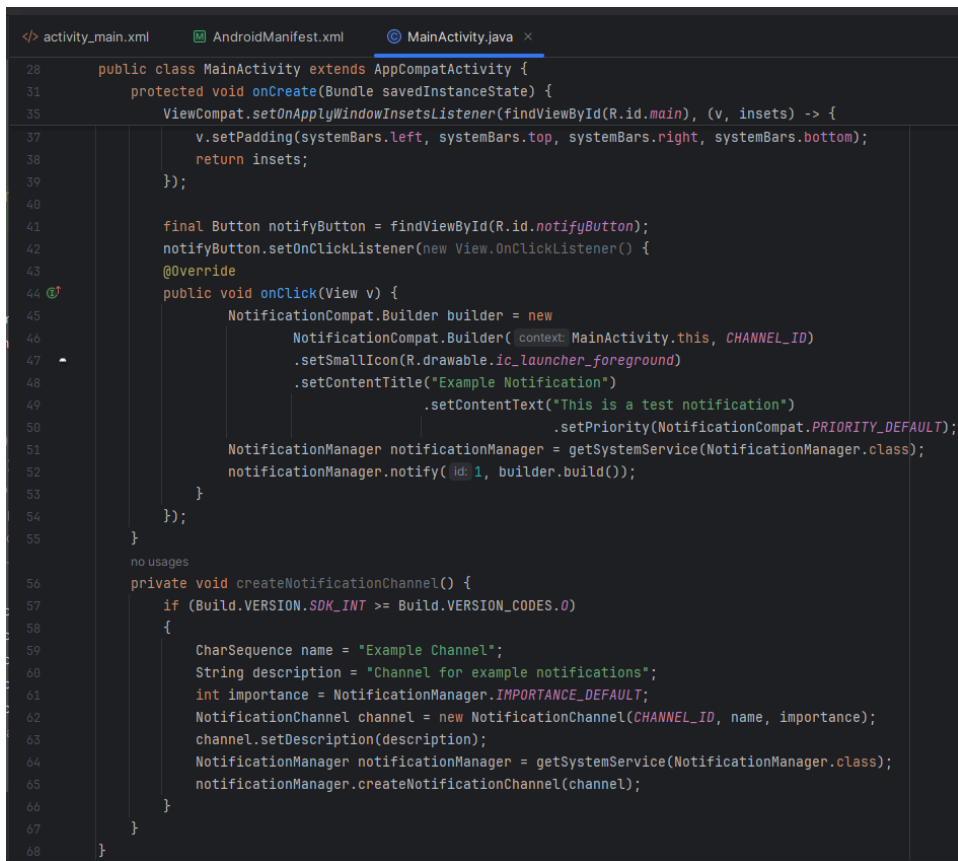
Уведомления в Android управляются через NotificationManager, который является системной службой, ответственной за доставку уведомлений. Для создания уведомления используется Notification.Builder класс, который позволяет настроить заголовок, текст, иконку и другие параметры уведомления. С Android Oreo (API уровень 26) введено понятие каналов уведомлений (notification channels), которое требует определения категории уведомлений, что позволяет пользователям управлять настройками уведомлений для различных типов контента.

Для отправки уведомлений на Android версии 13+ необходимо в файле манифеста прописать соответствующее разрешение (рисунок 1.4.1).

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

Рисунок 1.4.1 – Оформление разрешения в манифесте приложения на отправку уведомлений

Далее создадим код для показа простых уведомлений (рисунок 1.4.2).



```
28 public class MainActivity extends AppCompatActivity {
31     protected void onCreate(Bundle savedInstanceState) {
35         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
37             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
38             return insets;
39         });
40
41         final Button notifyButton = findViewById(R.id.notifyButton);
42         notifyButton.setOnClickListener(new View.OnClickListener() {
43             @Override
44             public void onClick(View v) {
45                 NotificationCompat.Builder builder = new
46                     NotificationCompat.Builder(context: MainActivity.this, CHANNEL_ID)
47                     .setSmallIcon(R.drawable.ic_launcher_foreground)
48                     .setContentTitle("Example Notification")
49                     .setContentText("This is a test notification")
50                     .setPriority(NotificationCompat.PRIORITY_DEFAULT);
51                 NotificationManager notificationManager = getSystemService(NotificationManager.class);
52                 notificationManager.notify(1, builder.build());
53             }
54         });
55     }
56
57     private void createNotificationChannel() {
58         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O)
59         {
60             CharSequence name = "Example Channel";
61             String description = "Channel for example notifications";
62             int importance = NotificationManager.IMPORTANCE_DEFAULT;
63             NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
64             channel.setDescription(description);
65             NotificationManager notificationManager = getSystemService(NotificationManager.class);
66             notificationManager.createNotificationChannel(channel);
67         }
68     }
69 }
```

Рисунок 1.4.2 – Логика отправки уведомления пользователю

В этом примере создается канал уведомлений, что необходимо для API 26 и выше. Затем, при нажатии на кнопку, создается уведомление с использованием NotificationCompat.Builder, которое отображает текст и иконку (рисунок 1.4.2). Уведомление отправляется через NotificationManager.

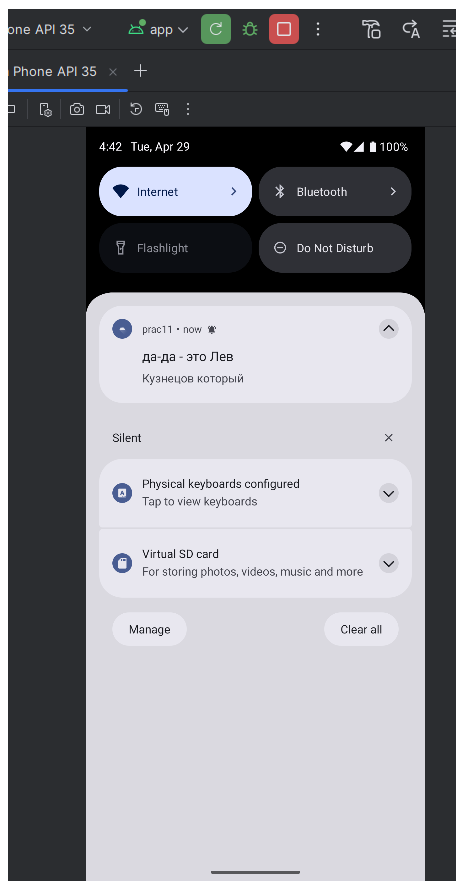


Рисунок 1.4.2 – Отображение отправленного уведомления

Используемая переменная CHANNEL_ID должна быть задана глобально в классе `private static final String CHANNEL_ID = "example_channel"`.

Отложенные уведомления в Android позволяют отправить уведомление через определённый период времени, что может быть полезно для напоминаний, задач по расписанию или для других сценариев, где требуется не немедленное, а запланированное уведомление. Для реализации отложенных уведомлений можно использовать AlarmManager, который позволяет запланировать выполнение кода в определённое время (рисунки 1.4.3 – 1.4.4).

```

33
34 <> public class MainActivity extends AppCompatActivity {
35     2 usages
36     public static final String CHANNEL_ID = "delayed_channel";
37     @Override
38     protected void onCreate(Bundle savedInstanceState) {
39         super.onCreate(savedInstanceState);
40         EdgeToEdge.enable(this);
41         setContentView(R.layout.activity_main);
42         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
43             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
44             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
45             return insets;
46         });
47         final Button delayedNotifyButton = findViewById(R.id.delayed_notify_button);
48         delayedNotifyButton.setOnClickListener(new View.OnClickListener() {
49             @Override
50             public void onClick(View v) {
51                 scheduleNotification(2000); // 2 секунда
52             }
53         });
54     }
55     no usages
56     private void createNotificationChannel() {
57         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
58             CharSequence name = "Walter?!";
59             String description = "Channel for delayed example notifications";
60             int importance = NotificationManager.IMPORTANCE_DEFAULT;
61             NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
62             channel.setDescription(description);
63             NotificationManager notificationManager = getSystemService(NotificationManager.class);
64             notificationManager.createNotificationChannel(channel);
65         }
66     }
67 }

```

Рисунок 1.4.3 – Часть 1 описания логики отправки сообщений с задержкой

```

34 public class MainActivity extends AppCompatActivity {
35 }
36
37 1 usage
38 private void scheduleNotification(Long delay) {
39     Intent notificationIntent = new Intent(this, AlarmReceiver.class);
40     PendingIntent pendingIntent = PendingIntent.getBroadcast(this, 0, notificationIntent,
41         PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
42     AlarmManager alarmManager = (AlarmManager) getSystemService(ALARM_SERVICE);
43     long futureInMillis = System.currentTimeMillis() + delay;
44     try {
45         alarmManager.setExact(AlarmManager.RTC_WAKEUP, futureInMillis, pendingIntent);
46     } catch (SecurityException e) {
47     }
48 }
49 }

```

Рисунок 1.4.4 - Часть 2 описания логики отправки сообщений с задержкой

Также для корректной работы приложения необходимо описать вспомогательный класс – AlarmReceiver (рисунок 1.4.5).

```

1 package com.example.prac11;
2
3 import android.content.BroadcastReceiver;
4 import android.content.Context;
5 import android.content.Intent;
6 import android.app.NotificationManager;
7 import androidx.core.app.NotificationCompat;
8
9
10 <> public class AlarmReceiver extends BroadcastReceiver {
11     @Override
12     public void onReceive(Context context, Intent intent) {
13         NotificationCompat.Builder builder = new
14             NotificationCompat.Builder(context, MainActivity.CHANNEL_ID)
15             .setSmallIcon(R.drawable.ic_launcher_foreground)
16             .setContentTitle("НАААААЙС! (delayed message)")
17             .setContentText("ФИШКИ РАБОТАЮТ!")
18             .setPriority(NotificationCompat.PRIORITY_DEFAULT);
19         NotificationManager notificationManager = (NotificationManager) context.getSystemService(Context.NOTIFICATION_SERVICE);
20         notificationManager.notify(1, builder.build());
21     }
22 }

```

Рисунок 1.4.5 – Описание логики класса AlarmReceiver

При нажатии на кнопку на экране через 2 секунды появляется оповещение (рисунок 1.4.6).

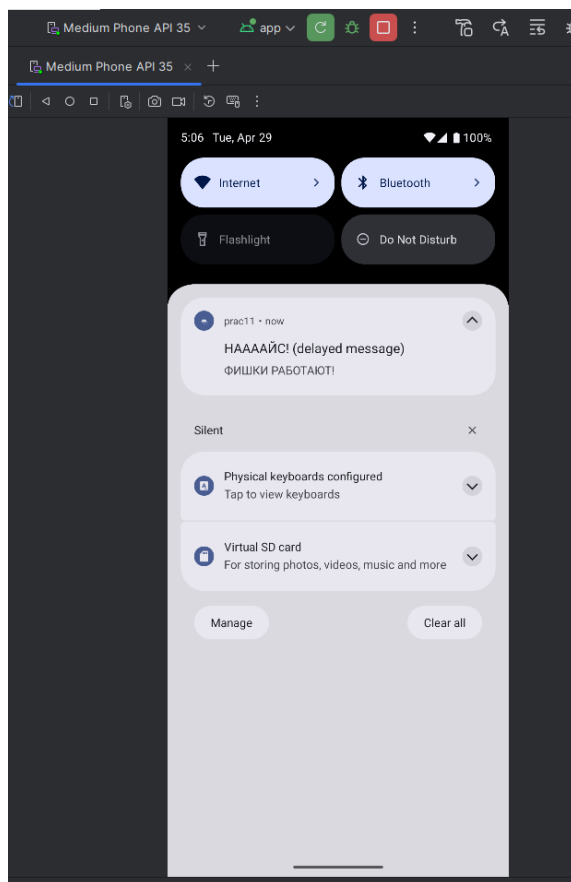
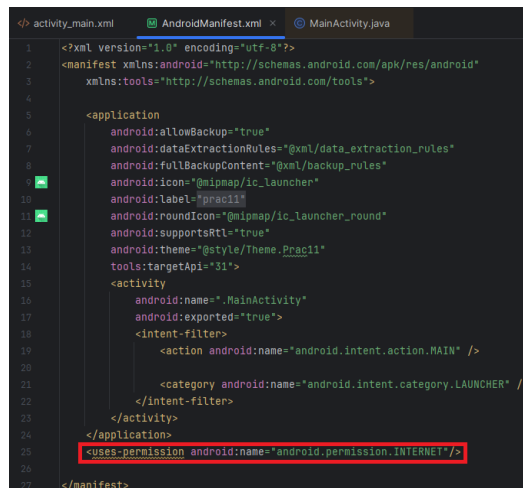


Рисунок 1.4.6 – Итоговый вид полученного оповещения

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Просмотр веб-страницы через WebView

Для просмотра страницы через WebView необходимо получить разрешение на доступ в интернет (рисунок 2.1.1).



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.AppCompat"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <uses-permission android:name="android.permission.INTERNET" />
    </application>
</manifest>
```

Рисунок 2.1.1 – Добавление разрешения на выход в интернет

Также необходимо описать файл разметки activity с использованием WebView (рисунок 2.1.2).

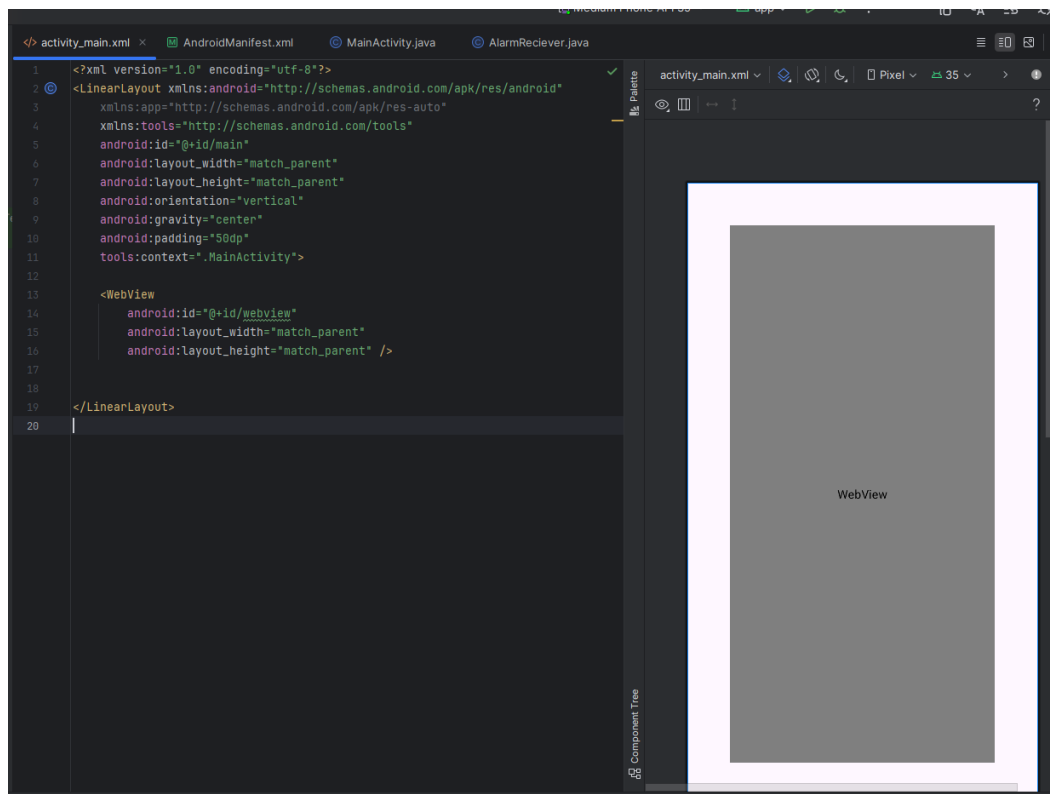


Рисунок 2.1.2 – Кодовый и графический вид файла разметки выбранной activity

Далее следует непосредственно описание логики использования WebView для просмотра веб-страницы (рисунок 2.1.3).

```
<> activity_main.xml  AndroidManifest.xml  MainActivity.java  AlarmReceiver.java
27  import androidx.core.app.NotificationCompat;
28  import androidx.core.graphics.Insets;
29  import androidx.core.view.ViewCompat;
30  import androidx.core.view.WindowInsetsCompat;
31
32  import java.io.IOException;
33
34  public class MainActivity extends AppCompatActivity {
35      1 usage
36      public static final String CHANNEL_ID = "delayed_channel";
37
38      4 usages
39      private WebView webView;
40
41      @Override
42      protected void onCreate(Bundle savedInstanceState) {
43          super.onCreate(savedInstanceState);
44          EdgeToEdge.enable(this);
45          setContentView(R.layout.activity_main);
46          ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
47              Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
48              v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
49              return insets;
50          });
51
52          webView = (WebView) findViewById(R.id.webview);
53          webView.setWebViewClient(new WebViewClient()); //Помогает приложению открывать ссылки внутри WebView, а не во внешнем браузере
54          webView.getSettings().setJavaScriptEnabled(true); //Включаем поддержку JavaScript
55
56          webView.loadUrl("https://supermangal.ru"); //Загрузка страницы
57      }
58  }
```

Рисунок 2.1.3 – Описание логики работы с WebView

На данном этапе необходимо проверить корректность работы написанного кода посредством запуска приложения (рисунок 2.1.4).

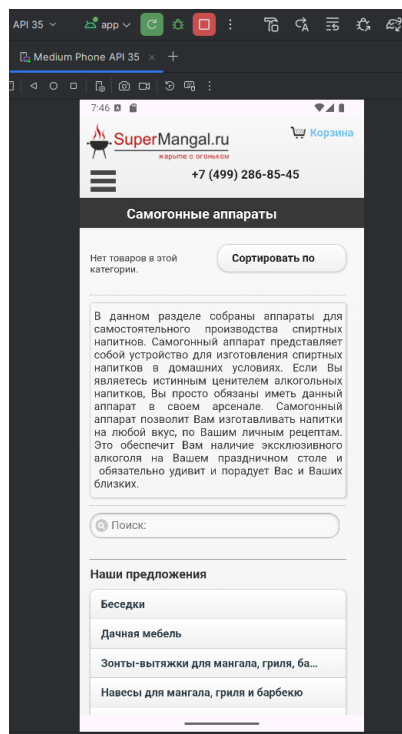


Рисунок 2.1.4 – Итоговый вид приложения

Как видно из рисунка 2.1.4 приложение отработывает без ошибок, значит, код работает корректно.

2.2 Реализация проигрывания музыки из интернета

Для начала необходимо описать файл разметки activity, где будет присутствовать кнопка, через нажатие которой будет осуществляться начало и остановка проигрывания музыки (рисунок 2.2.1).

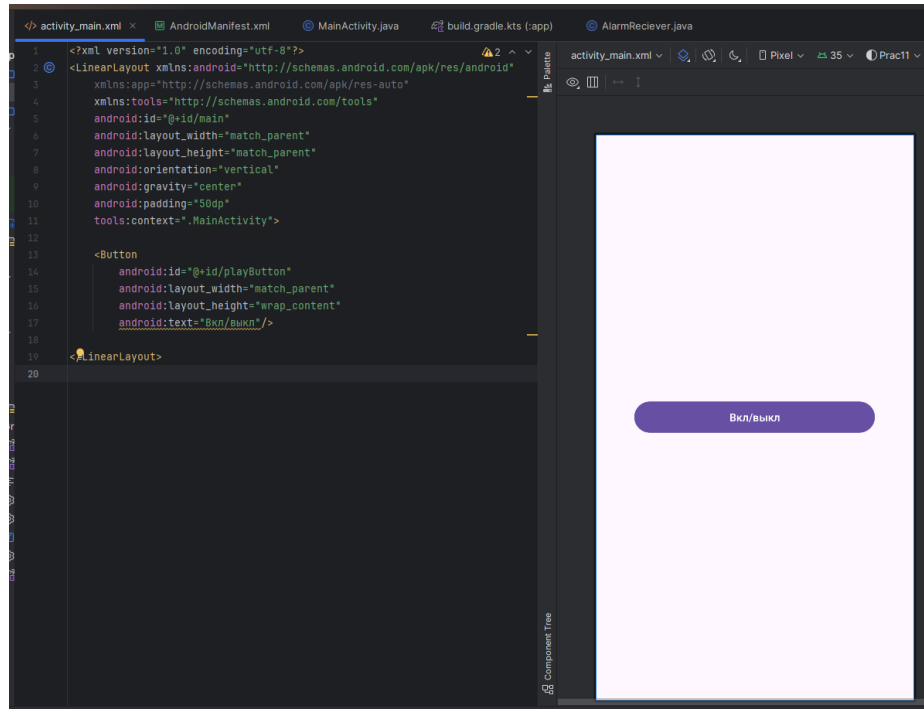


Рисунок 2.2.1 – Кодовый и графический вид файла разметки главной activity

Далее следует описание логики воспроизведения музыки (рисунки 2.2.2 – 2.2.3).

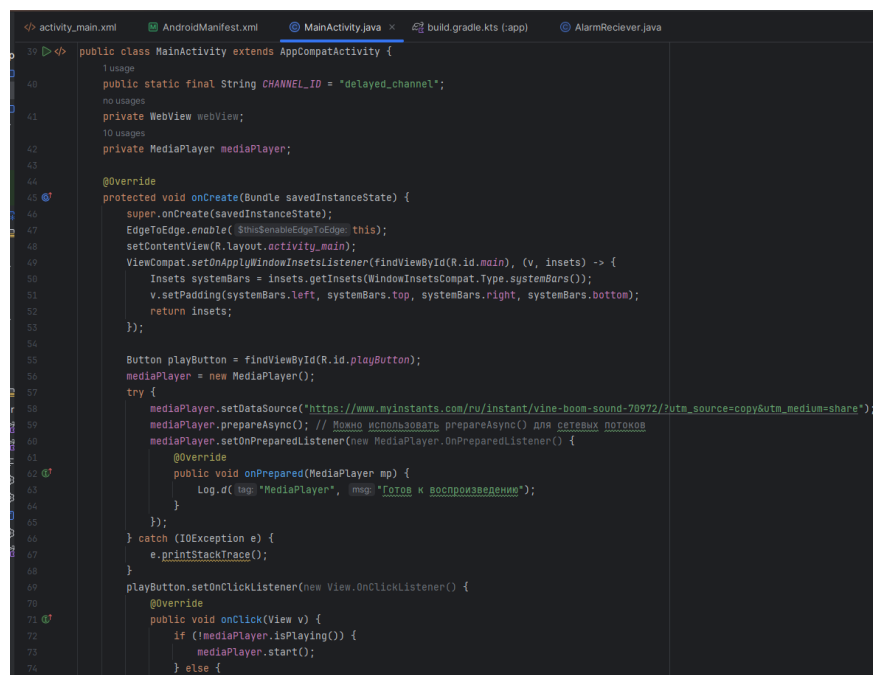


Рисунок 2.2.2 – Часть 1 описания логики воспроизведения музыки

```

69         playButton.setOnClickListener(new View.OnClickListener() {
70             @Override
71             public void onClick(View v) {
72                 if (!mediaPlayer.isPlaying()) {
73                     mediaPlayer.start();
74                 } else {
75                     mediaPlayer.pause();
76                 }
77             }
78         });
79     }
80     @Override
81     protected void onDestroy() {
82         super.onDestroy();
83         if (mediaPlayer != null) {
84             mediaPlayer.release();
85             mediaPlayer = null;
86         }
87     }
88 }

```

Рисунок 2.2.3 – Часть 2 описания логики воспроизведения музыки

На рисунках 2.2.2 – 2.2.3 отображено создание объекта класса MediaPlayer и последующее добавление слушателя событий на элемент разметки с описанием логики обработки события слушателем.

При нажатии на элемент разметки музыка начнёт или закончит воспроизведение, если она до этого не играла или играла соответственно (рисунок 2.2.4).

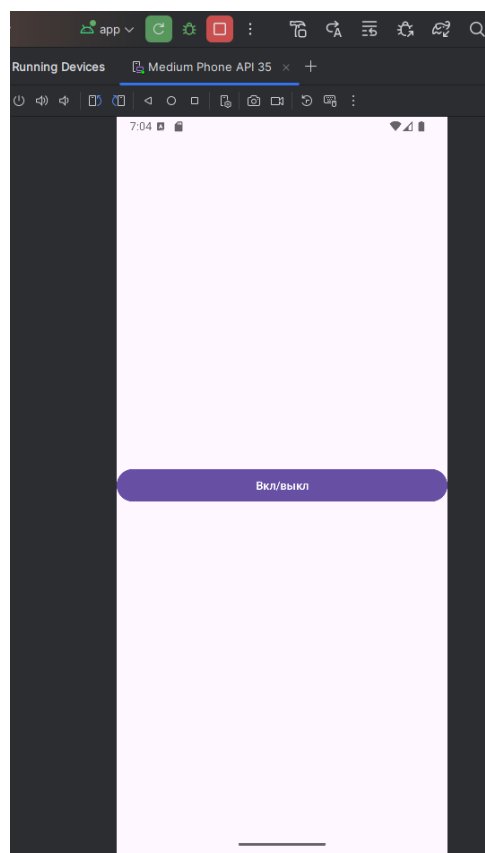


Рисунок 2.2.4 – Итоговый вид приложения

2.3 Реализация различных анимаций элементов UI

В качестве анимаций различных элементов будут использоваться вращение, перемещение и увеличение элементов разметки.

Для реализации перечисленных анимаций необходимо описать файл разметки главной activity (рисунок 2.3.1).

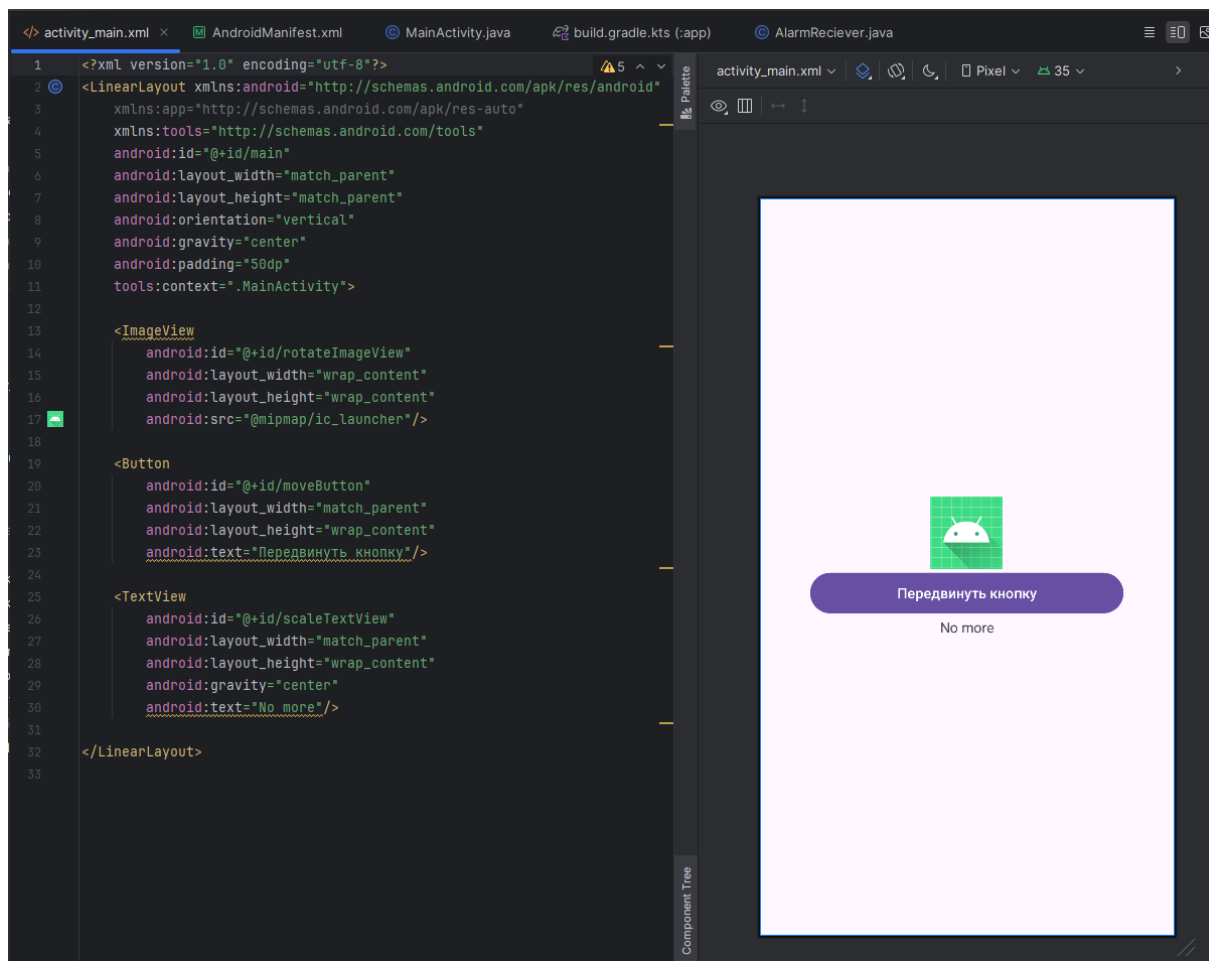


Рисунок 2.3.1 – Кодовый и графический вид файла разметки главной activity

Также необходимо описать логику анимаций согласно их элементам: вращение для ImageView, перемещение для Button и изменение размеров для TextView (рисунок 2.3.2).

```

39 public class MainActivity extends AppCompatActivity {
40
41     @Override
42     protected void onCreate(Bundle savedInstanceState) {
43         super.onCreate(savedInstanceState);
44         EdgeToEdge.enable(this);
45         setContentView(R.layout.activity_main);
46         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
47             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
48             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
49             return insets;
50         });
51
52         ImageView imageView = findViewById(R.id.rotateImageView);
53         ObjectAnimator rotateAnim = ObjectAnimator.ofFloat(imageView, "rotation", 0f, 360f);
54         rotateAnim.setDuration(2000);
55         rotateAnim.setRepeatCount(ObjectAnimator.INFINITE);
56         rotateAnim.setRepeatMode(ObjectAnimator.RESTART);
57         rotateAnim.start();
58
59         final Button moveButton = findViewById(R.id.moveButton);
60         moveButton.setOnClickListener(new View.OnClickListener() {
61             @Override
62             public void onClick(View v) {
63                 ObjectAnimator moveAnim = ObjectAnimator.ofFloat(moveButton, "translationX", 0f, 300f);
64                 moveAnim.setDuration(1000);
65                 moveAnim.start();
66             }
67         });
68
69         TextView scaleText = findViewById(R.id.scaleTextView);
70         scaleText.setOnClickListener(new View.OnClickListener() {
71             @Override
72             public void onClick(View v) {
73                 ObjectAnimator scaleX = ObjectAnimator.ofFloat(scaleText, "scaleX", 1f, 2f);
74                 ObjectAnimator scaleY = ObjectAnimator.ofFloat(scaleText, "scaleY", 1f, 2f);
75                 scaleX.setDuration(1000);
76                 scaleY.setDuration(1000);
77                 scaleX.start();
78                 scaleY.start();
79             }
80         });
81     }
82 }

```

Рисунок 2.3.2 – Описание логики работы анимаций

Далее необходимо удостовериться корректной работе приложения посредством запуска итогового приложения и взаимодействия с элементами (рисунки 2.3.3 – 2.3.4).

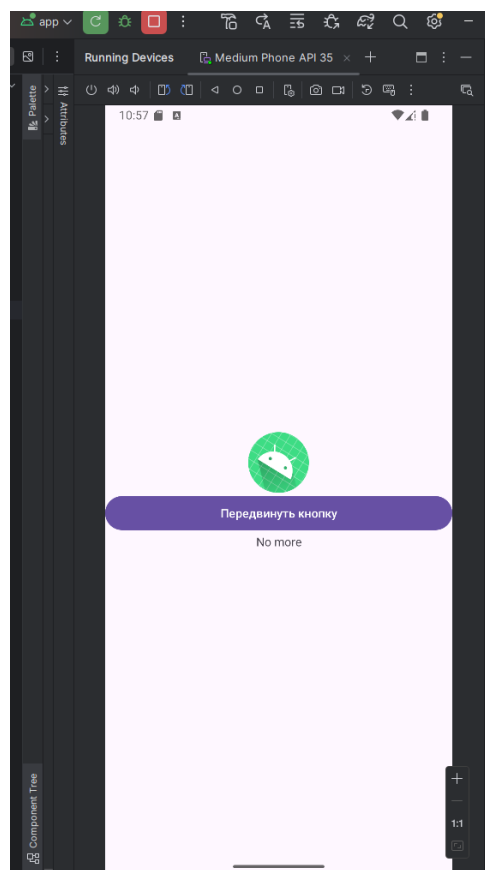


Рисунок 2.3.3 – Часть 1 отображения итогового вида приложения

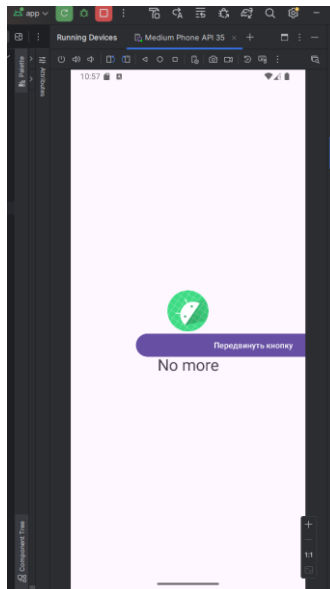


Рисунок 2.3.4 - Часть 2 отображения итогового вида приложения

2.4 Реализация отправки уведомлений: обычная и отложенная

Для работы с отложенными уведомлениями необходимо описать логику класса `AlarmReceiver`, который будет отвечать за отображение уведомления (рисунок 2.4.1).

```

1 package com.example.prac11;
2
3 import android.content.BroadcastReceiver;
4 import android.content.Context;
5 import android.content.Intent;
6 import android.app.NotificationManager;
7 import androidx.core.app.NotificationCompat;
8
9
10 </> public class AlarmReceiver extends BroadcastReceiver {
11     @Override
12     public void onReceive(Context context, Intent intent) {
13         NotificationCompat.Builder builder = new
14             NotificationCompat.Builder(context, MainActivity.CHANNEL_ID)
15             .setSmallIcon(R.drawable.ic_launcher_foreground)
16             .setContentTitle("ННААААЙС! (delayed message)")
17             .setContentText("ФИШКИ РАБОТАЮТ!")
18             .setPriority(NotificationCompat.PRIORITY_DEFAULT);
19         NotificationManager notificationManager = (NotificationManager) context.getSystemService(Context.NOTIFICATION_SERVICE);
20         notificationManager.notify(1, builder.build());
21     }
22 }

```

Рисунок 2.4.1 – Описание логики класса `AlarmReceiver`

Также необходимо описать файл разметки главной activity, где будут располагаться две кнопки: одна отвечает за отправку уведомлений, а другая за отправку уведомлений с задержкой (рисунок 2.4.2).

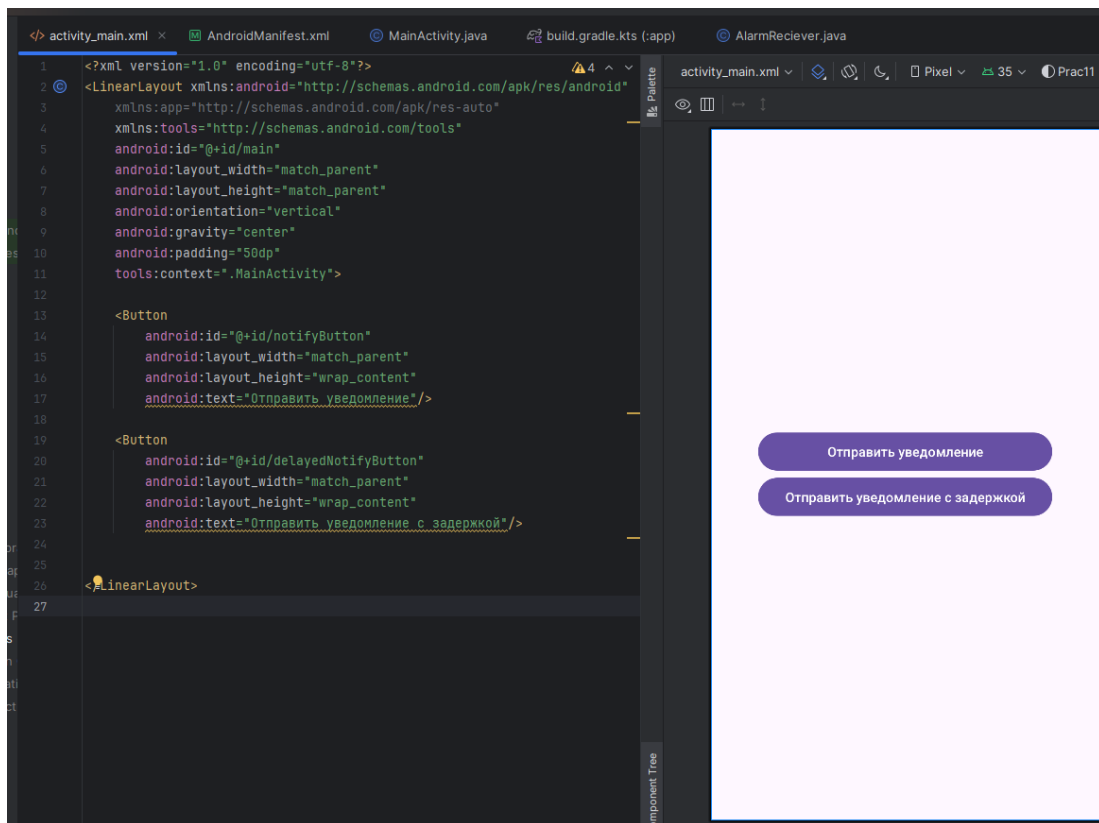


Рисунок 2.4.2 – Файл разметки главной activity

Далее необходимо описать логику обработки взаимодействий пользователя с GUI (рисунки 2.4.3 – 2.4.6).

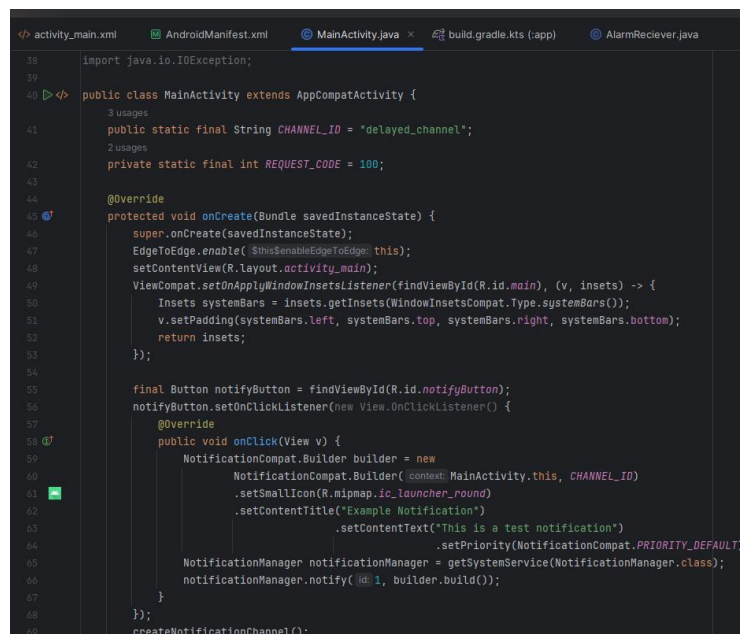


Рисунок 2.4.3 – Часть 1 описания логики работа главной activity

На рисунке 2.4.3 описывается отправка сообщения пользователю при нажатии на верхнюю кнопку.


```

40 public class MainActivity extends AppCompatActivity {
45     protected void onCreate(Bundle savedInstanceState) {
56         notifyButton.setOnClickListener(new View.OnClickListener() {
58             public void onClick(View v) {
67             }
68         });
69         createNotificationChannel();
70         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
71             requestPermissions(
72                 new String[]{Manifest.permission.POST_NOTIFICATIONS},
73                 REQUEST_CODE
74             );
75         }
76         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
77             if (!hasExactAlarmPermission(context.this)) {
78                 requestExactAlarmPermission(activity.this);
79             } else {
80                 Toast.makeText(context.this, text: "Разрешение уже есть!", Toast.LENGTH_SHORT).show();
81             }
82         }
83         final Button delayedNotifyButton = findViewById(R.id.delayedNotifyButton);
84         delayedNotifyButton.setOnClickListener(new View.OnClickListener() {
85             @Override
86             public void onClick(View v) {
87                 scheduleNotification(delay: 2); // 2 секунды
88             }
89         });
90     }

```

Рисунок 2.4.4 – Часть 2 описания логики работа главной activity

Чтобы можно было начать отправлять уведомления перед этим необходимо запросить у пользователя разрешение, что и описано в кодовом виде на рисунке 2.4.4. Также на этом рисунке описана отправка сообщения пользователю с задержкой при нажатии на нижнюю кнопку.

```

91 private void createNotificationChannel() {
92     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O)
93     {
94         CharSequence name = "Delayed Notifications";
95         String description = "Channel for delayed example notifications";
96         int importance = NotificationManager.IMPORTANCE_DEFAULT;
97         NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name, importance);
98         channel.setDescription(description);
99         NotificationManager notificationManager = getSystemService(NotificationManager.class);
100         notificationManager.createNotificationChannel(channel);
101     }
102 }
103
104 private void scheduleNotification(long delay) {
105     Intent notificationIntent = new Intent(packageContext.this, AlarmReciever.class);
106     PendingIntent pendingIntent =
107         PendingIntent.getBroadcast(context.this, requestCode: 0, notificationIntent, flags: PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
108     AlarmManager alarmManager = (AlarmManager)
109         getSystemService(ALARM_SERVICE);
110     long futureInMillis = System.currentTimeMillis() + delay;
111     try {
112         alarmManager.setExact(AlarmManager.RTC_WAKEUP, futureInMillis, pendingIntent);
113     } catch (SecurityException e) {
114         Toast.makeText(context.this, text: "none of it matters", Toast.LENGTH_LONG).show();
115     }
116 }
117
118 @Override
119 public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
120     super.onRequestPermissionsResult(requestCode, permissions, grantResults);
121     if (requestCode == REQUEST_CODE) {
122         if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
123             Toast.makeText(context.this, text: "Уведомления разрешены!", Toast.LENGTH_SHORT).show();
124         } else {
125             Toast.makeText(context.this, text: "Уведомления запрещены!", Toast.LENGTH_SHORT).show();
126         }
127     }
128 }

```

Рисунок 2.4.5 – Часть 3 описания логики работы главной activity

На рисунке 2.4.5 описана логика отправки сообщения с задержкой: в случае отказа на отправку пользователю высветится небольшое сообщение, уведомляющее его об этом.

```
<? activity_main.xml  AndroidManifest.xml  MainActivity.java  build.gradle.kts (.app)  AlarmReceiver.java
40  public class MainActivity extends AppCompatActivity {
    1 usage
    128  private void requestExactAlarmPermission(Activity activity) {
    129      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    130          Intent intent = new Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM);
    131          activity.startActivity(intent);
    132      }
    133  }
    134
    1 usage
    135  private boolean hasExactAlarmPermission(Context context) {
    136      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    137          AlarmManager alarmManager = (AlarmManager) context.getSystemService(Context.ALARM_SERVICE);
    138          return alarmManager.canScheduleExactAlarms();
    139      }
    140      return true;
    141  }
    142
    143  }
```

Рисунок 2.4.6 – Часть 4 описания логики работа главной activity

На рисунке 2.4.6 описаны методы запроса разрешения на отправку пользователю сообщений.

Далее необходимо убедиться в корректной работе данного приложения (рисунки 2.4.7 – 2.4.11).

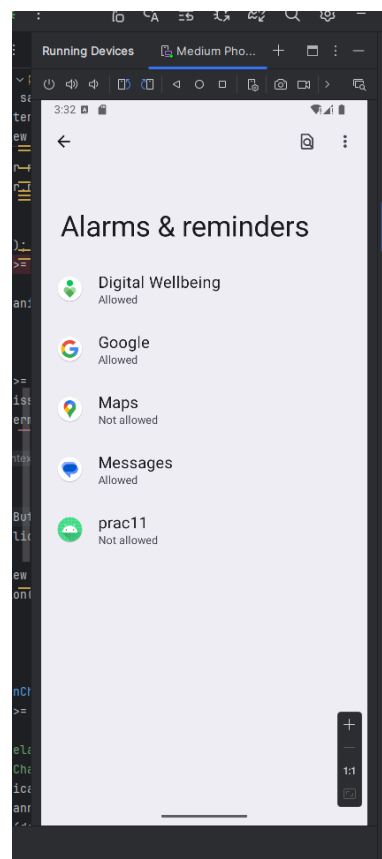


Рисунок 2.4.7 – Часть 1 отображения итогового вида приложения

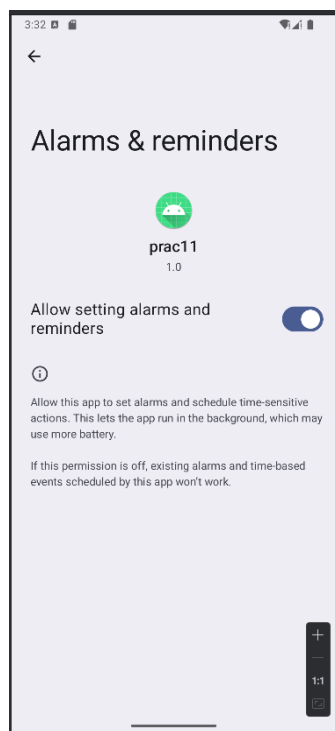


Рисунок 2.4.8 - Часть 2 отображения итогового вида приложения

Если у пользователя будет отключено разрешение на получение уведомлений от созданного приложения, то перед началом работы приложения будет проведён запрос по получения данного разрешения, что и отображено на рисунках 2.4.7 – 2.4.8.

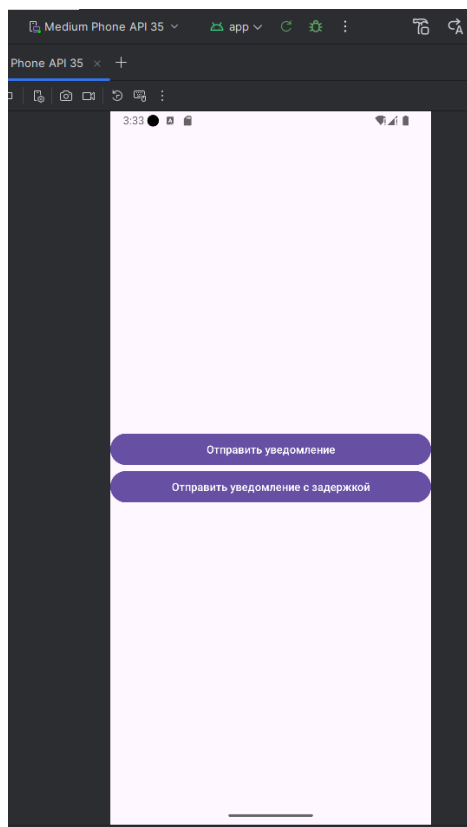


Рисунок 2.4.9 - Часть 3 отображения итогового вида приложения

При нажатии на кнопки, отображённые на рисунке 2.4.9 пользователю придёт уведомление (рисунок 2.4.10) и уведомление с задержкой в две секунды (рисунок 2.4.11) соответственно.

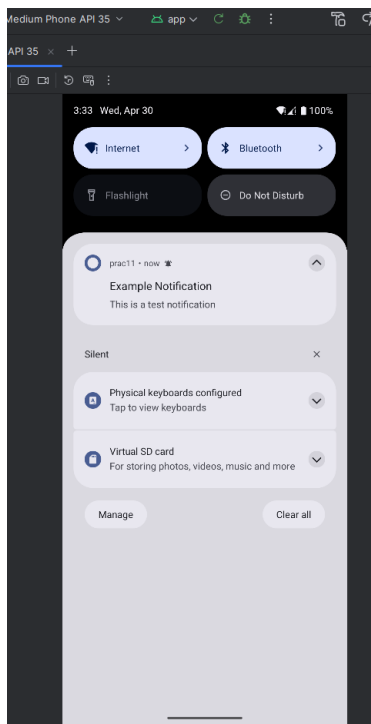


Рисунок 2.4.10 - Часть 4 отображения итогового вида приложения

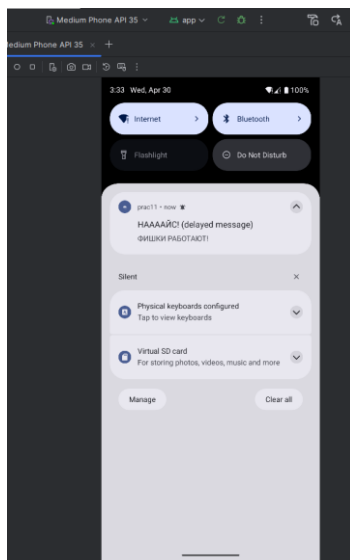


Рисунок 2.4.11 – Часть 5 отображения итогового вида приложения

Уведомления отображаются без ошибок, что говорит о корректной работе приложения.

ЗАКЛЮЧЕНИЕ

В ходе работы были получены знания по работе с WebView, анимациями и отправкой сообщений в Android Studio. Полученные знания были закреплены путём выполнения практического задания.